



Programrendszerek fejlesztése

10. gyakorlat

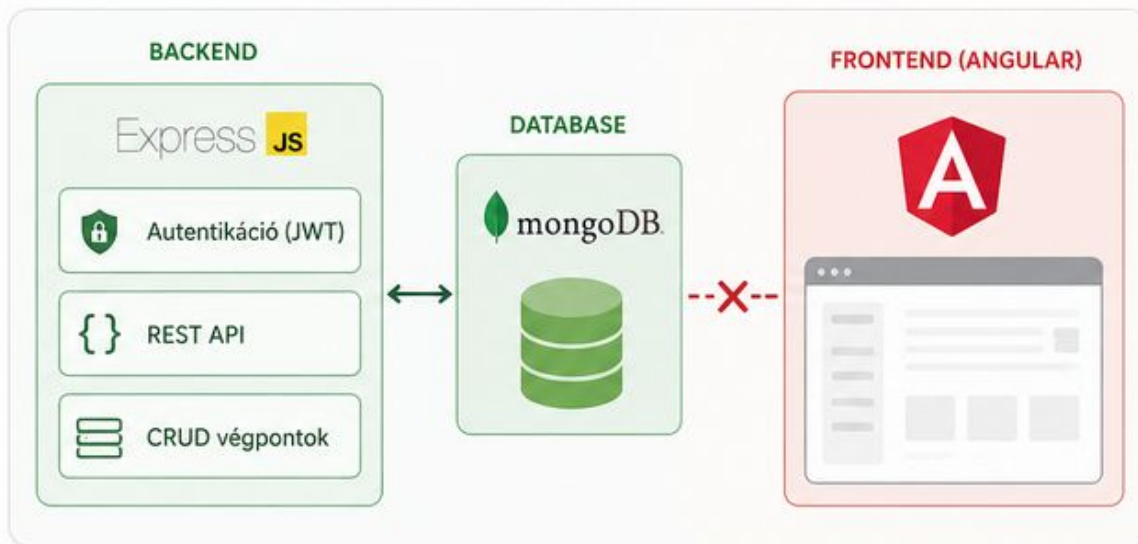
Dr. Jánki Zoltán Richárd

Hol tartunk most?



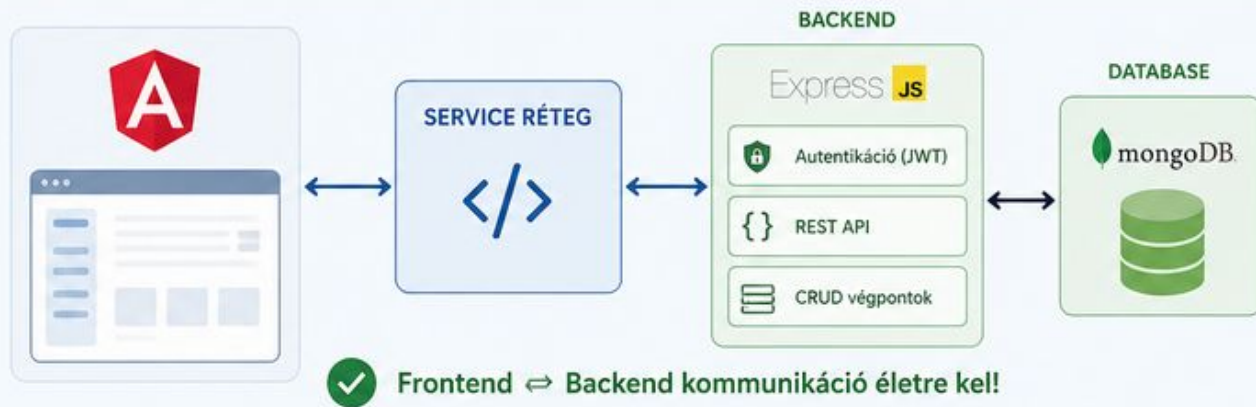
RÖVID ÁTTEKINTÉS

- ✓ A félév során felépítettünk egy Express.js REST API-t MongoDB-vel,
- ✓ autentikációval (JWT) és
- ✓ CRUD végpontokkal.
- ✓ Az Angular alkalmazásunk UI-ja kész, de „vak” — nem kommunikál a szerverrel.



MAI CÉL

Életre kelteni a frontend-et a **service réteg** bevezetésével.



Separation of Concerns (SoC)

✗ NEM AJÁNLOTT: HTTP hívás a komponensben

KOMPONENS

UserListComponent

```
export class UserListComponent {
  users: User[] = [];

  constructor(private http: HttpClient) {}

  ngOnInit() {
    this.http.get<User[]>('/api/users')
      .subscribe(data => this.users = data);
  }
}
```



- Összemossa a megjelenítést és az adathozzáférést
- Nehezen tesztelhető
- Új adatforrás váltása bonyolult (REST → WebSocket, cache, mock...)
- Újrafelhasználás nehézkes

✓ AJÁNLOTT: Service réteg használata

KOMPONENS (csak megjelenítés)

UserListComponent

USER SERVICE (adatforrás-kezelés)

getUsers(): Observable<User[]>



ADATFORRÁSOK



- ✓ A komponens felelőssége: megjelenítés
- ✓ Az adatforrás-kezelés elkülönítve
- ✓ Könnyen tesztelhető (service mockolható)
- ✓ Új adatforrás könnyen beépíthető (REST, WebSocket, cache, mock...)
- ✓ Újrafelhasználható service több komponensben



CÉL

Tiszta felelősségi körök,
könnyen karbantartható
és rugalmas alkalmazás.



Megjelenítés
a komponens feladata



Adatforrás-kezelés
a service feladata



Adatforrás váltása
egyszerű



Separation of concerns
a gyakorlatban

Függőségek injektálása

@Injectable() és a Dependency Injection

Az @Injectable() dekorátor egy osztályt szolgáltatássá (service) tesz, a DI rendszer pedig gondoskodik az előállításáról és megosztásáról.

SERVICE

```
@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private token: string | null = null;

  login(token: string) {
    this.token = token;
  }

  getToken(): string | null {
    return this.token;
  }
}
```

Mit jelent a **providedIn: 'root'**?

- ✓ A service a gyökér (root) injector-be kerül.
- ✓ Az alkalmazás teljes élettartama alatt létezik.
- ✓ Egyetlen példány (singleton) jön létre belőle.

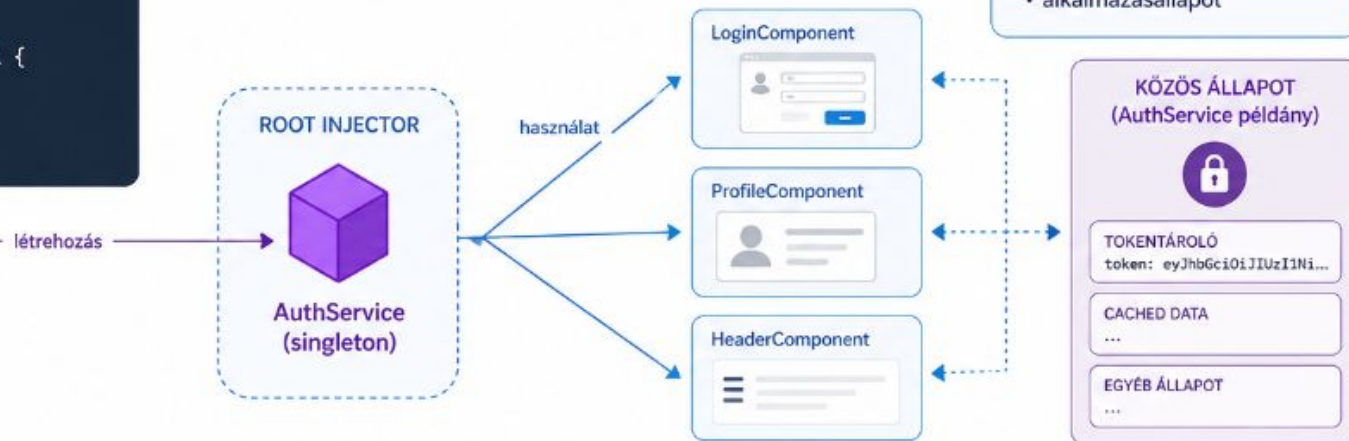


Miért singleton?

Az állapot konzisztens marad az összes komponensben.

Példák:

- auth token
- cached data
- felhasználói beállítások
- alkalmazásállapot



ÖSSZEFOGLALVA



@Injectable()

A szolgáltatás regisztrálása a DI rendszer számára.



providedIn: 'root'

A root injector biztosítja, hogy egyetlen példány létezzen.



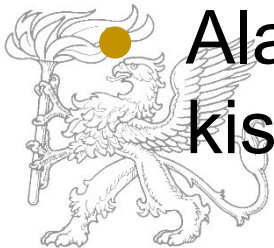
Singleton előny

Konzisztens állapot, megosztás az egész alkalmazásban.

HTTP kérés-válasz

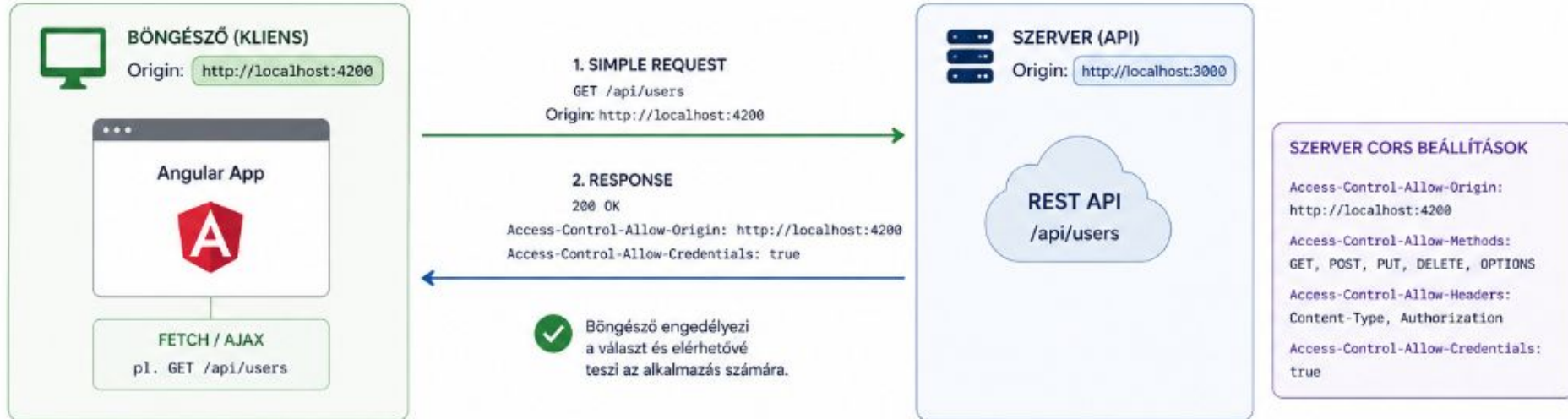
- HttpClientModule → Observable elven
- Tipikus CRUD függvények a service-ekben
 - getAll()
 - getById(id)
 - create(data)
 - update(id, data)
 - delete(id)

- Alap URL-t érdemes környezeti változóba kiszervezni



CORS működése

A böngésző biztonsági mechanizmusa, amely engedélyezi, hogy egy weboldal másik domainről származó erőforrást érjen el.



PRE-FLIGHT KÉRÉS

3. PRE-FLIGHT (ha nem simple request)
OPTIONS /api/users
Origin: `http://localhost:4200`
Access-Control-Request-Method: `POST`
Access-Control-Request-Headers: `Content-Type, Authorization`

4. PRE-FLIGHT RESPONSE

204 No Content
Access-Control-Allow-Origin: `http://localhost:4200`
Access-Control-Allow-Methods: `GET, POST, PUT, DELETE, OPTIONS`
Access-Control-Allow-Headers: `Content-Type, Authorization`
Access-Control-Allow-Credentials: `true`

A böngésző először ellenőrzi, hogy a szerver engedélyezi-e a kérést (metódus, fejléc, hitelesítés stb.).

MIÉRT VAN SZÜKSÉG CORS-RA?



- ✓ Védi a felhasználókat a jogosulatlan kérésektől.
- ✓ A böngésző alapértelmezetten blokkolja a cross-origin kéréseket.
- ✓ A CORS lehetővé teszi a kontrollált kivételeket.

MI SZÁMÍT ORIGIN-NEK?

scheme + host + port

`http://localhost:4200`

≠

`http://localhost:3000`

SIMPLE REQUEST FELTÉTELEI

- ✓ GET, HEAD vagy POST metódus
- ✓ Engedélyezett fejlécek (pl. Accept, Content-Type)
- ✓ Content-Type: `application/x-www-form-urlencoded`, `multipart/form-data` vagy `text/plain`

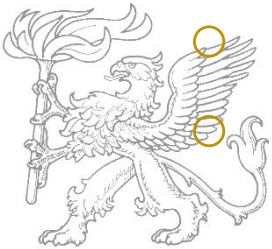


ESETLEGES HIBÁK

"No 'Access-Control-Allow-Origin' header is present on the requested resource."

Autentikáció kezelése

- AuthService felelőssége
 - login
 - logout
 - regisztráció
 - token tárolás
- JWT tárolása a localStorage-ban
- JWT hozzáadása a kéréshez
 - minden kéréshez hozzáadjuk
 - HttpInterceptor (kliens-oldali “middleware”)



További kérések kezelése

- A megfelelő service felelőssége
 - CRUD műveletek
 - további műveletek
- Fel-leiratkozás a komponensben
 - Service-nek ennek megfelelő értéket kell visszaadnia
- Hibakezelés interceptor segítségével



Gyakorlat

- Kössük össze a frontend-et a backend-del!
 - A bejelentkezést és regisztrációt szolgáljuk ki a backend-del!
 - Cseréljük le a teljes in-memory db-t service hívásokra!
 - Figyeljünk a CORS beállításokra!

